Listing 2.5 Adding the Welcome page to the main navigation

```
<ul class="navbar-nav flex-grow-1">
    <li class="nav-item">
        <a class="nav-link text-dark" asp-area="" asp-page="/Index">Home</a>
    </li>
    <li class="nav-item">
        <a class="nav-link text-dark" asp-area="" asp-page="/Privacy">Privacy</a>
    </li>
    <li class="nav-item">
        <a class="nav-link text-dark" asp-area="" asp-page="/Welcome">Welcome</a>
    </li>
</ul>
```

Refresh the browser again; the navigation menu at the top of every page will now include a link to the Welcome page. The change you just made has been applied to every page in the application. That is because you changed the layout file, which is used by all pages in the application. The content of the Razor page is merged with the content in the layout page to produce the final output.

You may wonder why there is no `href` attribute on the anchor element you added to the layout page to create the link. This element is called an anchor tag helper. *Tag helpers* are components that target regular HTML elements and enable server-side code to influence how they are rendered to the browser via special attributes that usually start with `asp-`. For example, the `asp-page` attribute takes a value representing the name of the page you want to generate a link to. Tag helpers are covered in more detail in the next chapter.

So you have seen some ways C# and HTML work together in the Razor page to produce HTML. Generally, the best advice is to restrict the amount of C# code you include in the Razor page to only that required to influence the presentation. Application logic, including algorithms that determine the time, should be kept out of the Razor page file. The first level of separation between the Razor page file and application logic is the `PageModel`